Fine-Tuning Small Language Models

A Practical Guide to Getting Started

Adapt a compact model to one job. Ship faster. Run cheaper. Keep data closer.

Base model

Pre-trained encoder with general language knowledge.

→

Task data

Labeled examples for billing, tech, or sales.

→

Tuned router

Learns one decision boundary extremely well.

Definition

Fine-tuning continues training a pre-trained model on a smaller, labeled dataset so it becomes excellent at a specific task.

Why “small” works

For focused supervised workflows, an SLM can be the practical sweet spot between quality, cost, deployability, and privacy.

Who Am I?

Engineering, Infrastructure, and Applied AI

Rahul Tripathi

Director of Infrastructure Engineering, Scopely

Experience & Expertise

~30 years in Software & Infrastructure Engineering and Leadership

Academic Pursuits

- M.S. in A.I. Candidate, UT Austin
- Bachelors in CS (IITK ‘97)

Focus: Practical, scalable, and cost-effective AI

02 Why choose an SLM over an LLM API?

The business case: cost, latency, and privacy

Running example: automated support ticket routing

Classify each message as Billing, Tech Support, or Sales.

Cost efficiency

Massive LLMs are often overkill for repetitive, high-volume classification. A fine-tuned SLM can do the same narrow routing job at much lower per-request cost.

100k tickets/day

Latency & edge deployment

API calls add network hops. A local model can sit beside the app server and return a route in milliseconds instead of waiting on round-trips.

live chat

Data privacy

Support tickets may include passwords, account IDs, or PII. Open-weights deployment lets routing stay on-prem or inside a controlled VPC.

PII aware

Decision rule

If the task is stable, labels are known, and throughput matters, an SLM usually beats a general-purpose API on operational fit.

03

The hardware myth

You do not need a supercomputer

Training from scratch

Building a foundation model requires server farms, long runs, and enormous data.

Fine-tuning an SLM

You are nudging an already-capable encoder toward your label space, not teaching language from zero.

Starter setup

A consumer GPU, modest batch sizes, and a short iteration loop are often enough to get a practical classifier working.

Efficiency strategy: Freeze weights

Update only the task-specific head or a small adapter. This slashes memory use and compute, making local experimentation feasible.

Practical takeaway: Meaningful fine-tuning can be done locally on standard consumer hardware.

04 The Hugging Face pipeline, and a demo

How it actually works under the hood

1 SELECT MODEL → 2 TOKENIZE → 3 LABEL DATA → 4 TRAINER → 5 EVALUATE

```
model = AutoModelForSequenceClassification.from_pretrained(
MODEL_NAME, num_labels=3)
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
...
trainer = Trainer(
model=model,
args=training_args,
train_dataset=train_ds
)
trainer.train()
```

What each part does

The Hub gives you a pretrained checkpoint. The tokenizer converts raw text into token IDs. The Trainer manages batching, loss, metrics, evaluation, and checkpoints.

** In ML terms, routing a ticket to one of three queues is a Sequence Classification task.*

Mental model

You mainly prepare the model, the tokenizer, and a labeled dataset. Then you let the training loop do the repetitive work.

05

The lazy model problem

A lesson in spurious correlations

Principle

Empirical risk minimization means the model will take the easiest path that lowers error.

Shortcut artifact

If “**invoice**” strongly correlates with **Billing**, the model may stop reading the rest of the sentence.

Core lesson

The problem is usually not that the model is irrational. The problem is that the data rewarded a cheap shortcut.

Failure example

“The invoice page crashes every time I click download.”

Baseline model

Sees “invoice” -> routes to Billing ❌

What we wanted

Read the whole request -> route to Tech Support ✅

Fixing the data, not the model

Why data distribution matters most

Bad instinct

Swap architectures too early and hope a different backbone will magically understand the edge case.

Better instinct

Change what the model sees during training.

1 Find hard examples

Collect tickets where the obvious keyword shortcut fails.

2 Up-weight them

Duplicate or increase their influence during training.

3 Force semantic learning

Now the shortcut is no longer enough to minimize loss.

07

Another Demo!

A/B testing inference for ticket routing

Setup

Compares a Baseline (Lazy) model and a Debiased (Robust) model side by side.

“My subscription renewal charged the wrong credit card”

Route

Baseline (Lazy)

Prediction: Sales ❌

- Uses the trigger word “subscription”
- Ignores billing context

Debiased (Robust)

Prediction: Billing ✅

- Uses payment intent
- Reads the full request

Inference on this 14M parameter model takes ~20ms locally. Cloud API calls take ~300ms.

Key takeaways for practitioners

Start small, focus on data

01

Start with local experiments

Use Hugging Face and a compact model to validate the workflow before you spend on heavier infrastructure.

02

Treat data as the main lever

A fine-tuned model is bounded by the quality and distribution of the data that trained it.

03

Evaluate for robustness

A better validation score can still hide brittle behavior. Test edge cases, shortcuts, and adversarial phrasing explicitly.

Bottom line: for focused supervised tasks, the fastest path to a useful model is usually smaller model + better data.